



RAPPORT DE PROJET DE FIN DE MODULE

Module : Deep Learning

Conception, implémentation, comparaison et analyse critique de modèles de Deep Learning pour données tabulaires, images et séquences

Filière / Département : Informatique ia et data

Réalisée par : Amira ZAHR

Année universitaire 2025–2026

Sommaire

INTRODUCTION GÉNÉRALE

PARTIE I — PERCEPTRON MULTICOUCHE (MLP) SUR DONNÉES TABULAIRES

- [1.1 Rappels théoriques](#)
- [1.2 Description des données](#)
- [1.3 Exploration visuelle des données \(EDA\)](#)
- [1.4 Prétraitement des données](#)
- [1.5 Architecture MLP — deux implémentations](#)
- [1.6 Stratégies d'initialisation](#)
- [1.7 Entraînement complet](#)
- [1.8 Sauvegarde et rechargement du modèle](#)
- [1.9 Évaluation finale sur le jeu de test](#)
- [1.10 Analyse critique](#)
- [1.11 Question de synthèse — Partie I](#)

PARTIE II — RÉSEAU DE NEURONES CONVOLUTIONNEL (CNN) SUR IMAGES

- [2.1 Pourquoi un CNN plutôt qu'un MLP pour les images](#)
- [2.2 Description des données](#)
- [2.3 Exploration visuelle des données](#)
- [2.4 Calculs manuels — tailles de sortie convolution et pooling](#)
- [2.5 Implémentations manuelles vs couches PyTorch](#)
- [2.6 Architecture CNN — inspirée de LeNet](#)
- [2.7 Étude d'ablation — influence des choix architecturaux](#)
- [2.8 Visualisation des cartes de caractéristiques \(feature maps\)](#)
- [2.9 Comparaison MLP vs CNN sur le même dataset](#)
- [2.10 Évaluation finale du CNN — matrice de confusion](#)
- [2.11 Analyse critique](#)
- [2.12 Question de synthèse — Partie II](#)

PARTIE III — RNN, LSTM, GRU ET SEQ2SEQ

- [3.1 Rappels théoriques](#)
- [Volet A — Modèle de langage \(niveau caractère\)](#)
 - [3.2 Description des données](#)
 - [3.3 Visualisation et exploration](#)
 - [3.4 Prétraitement](#)
 - [3.5 Architectures RNN, LSTM, GRU](#)

[3.6 Entraînement \(BPTT et gradient clipping\)](#)

[3.7 Évaluation et comparaison](#)

[3.8 Génération de texte qualitative](#)

[3.9 Synthèse — Volet A](#)

[Volet B — Seq2Seq : traduction automatique](#)

[3.10 Description des données](#)

[3.11 Visualisation et filtrage](#)

[3.12 Prétraitement](#)

[3.13 Architecture encodeur-décodeur](#)

[3.14 Entraînement](#)

[3.15 Évaluation : décodage glouton vs beam search, score BLEU](#)

[3.16 Synthèse — Volet B](#)

[3.17 Synthèse finale — Partie III](#)

[SYNTHÈSE GÉNÉRALE ET CONCLUSION DU PROJET](#)

[Tableau comparatif des trois familles d'architectures](#)

[Discussion transversale](#)

[Limites globales et perspectives](#)

Liste des figures

[Figure 1 — Distribution de la variable cible, de l'âge, des heures travaillées et du niveau d'études encodé, selon le revenu.](#)

[Figure 2 — Niveau d'études, occupation \(top 8\), sexe et autres variables catégorielles croisées avec le revenu.](#)

[Figure 3 — Courbes de perte \(train/validation\) et de métriques \(exactitude/F1\) de validation sur 40 époques.](#)

[Figure 4 — Matrice de confusion du MLP sur le jeu de test \(exactitude = 0,829, F1 = 0,699\).](#)

[Figure 5 — Un exemple représentatif de chacune des 10 classes du jeu de données Fashion-MNIST.](#)

[Figure 6 — Distribution des classes \(parfaitement équilibrée\) et distribution des intensités de pixels \(bimodale\).](#)

[Figure 7 — Cartes de caractéristiques des couches conv1 \(haut\) et conv2 \(bas\) pour une image de test \(classe « Shirt »\).](#)

[Figure 8 — Courbes de perte d'entraînement et d'exactitude de validation — MLP vs CNN \(LeNet\).](#)

[Figure 9 — Matrice de confusion du CNN \(LeNet\) sur le jeu de test Fashion-MNIST \(exactitude = 0,854\).](#)

[Figure 10 — Fréquence des 20 caractères les plus fréquents du corpus utilisé pour le Volet A.](#)

[Figure 11 — Perplexité de validation par époque — comparaison RNN / LSTM / GRU.](#)

[Figure 12 — Distribution des longueurs de phrases \(en mots\) pour l'anglais et le français — corpus de repli.](#)

Liste des tableaux

[Tableau 1 — Description générale du jeu de données Adult Income.](#)

[Tableau 2 — Dimensions des ensembles d'entraînement, de validation et de test après prétraitement.](#)

[Tableau 3 — Comparaison des paramètres nommés \(named parameters\) entre les deux implémentations du MLP.](#)

[Tableau 4 — Comparaison de trois stratégies d'initialisation des poids \(15 époques d'entraînement court\).](#)

[Tableau 5 — Évolution des métriques d'entraînement et de validation par paliers de 5 époques \(meilleur F1 de validation : 0,7012\).](#)

[Tableau 6 — Métriques d'évaluation finales du MLP sur le jeu de test \(6 777 observations\).](#)

[Tableau 7 — Description générale du jeu de données Fashion-MNIST.](#)

[Tableau 8 — Vérification numérique des formules de taille de sortie pour la convolution et le pooling.](#)

[Tableau 9 — Comparaison des implémentations manuelles et des couches PyTorch natives : résultats identiques.](#)

[Tableau 10 — Détail des paramètres de l'architecture LeNet \(total : 61 706 paramètres\).](#)

[Tableau 11 — Étude d'ablation : influence du padding, du stride, du type de pooling, du nombre de filtres et d'une convolution 1×1.](#)

[Tableau 12 — Comparaison finale MLP vs CNN sur Fashion-MNIST \(même nombre d'époques\).](#)

[Tableau 13 — Nombre de paramètres des trois architectures du Volet A \(même embed_dim et hidden_dim\).](#)

[Tableau 14 — Évolution de l'entraînement \(perte, perplexité de validation, norme du gradient\) pour RNN, LSTM et GRU sur 3 époques.](#)

[Tableau 15 — Perplexité finale sur le jeu de test, nombre de paramètres et temps total d'entraînement \(Volet A\).](#)

[Tableau 16 — Évolution de l'entraînement Seq2Seq sur 6 époques \(teacher_forcing_ratio = 0,5\).](#)

[Tableau 17 — Score BLEU moyen sur le jeu de test, décodage glouton vs beam search.](#)

[Tableau 18 — Tableau comparatif synthétique des trois familles d'architectures étudiées dans le projet.](#)

Introduction générale

Ce rapport constitue le livrable final du module Deep Learning (EMSI Casablanca, année universitaire 2025–2026). Conformément au sujet du projet, il couvre trois familles d'architectures de deep learning appliquées à trois types de données distincts : des données tabulaires (Partie I — perceptron multicouche), des images (Partie II — réseau de neurones convolutionnel) et des séquences textuelles (Partie III — réseaux récurrents et architecture encodeur-décodeur). L'objectif transversal du projet est de montrer que le choix d'une architecture de deep learning découle directement de la structure des données traitées : absence de structure spatiale/temporelle pour le tabulaire, structure spatiale locale pour l'image, structure séquentielle et dépendance temporelle pour le texte.

Chaque partie suit la même progression méthodologique imposée par le sujet : rappel des fondements théoriques, description et exploration du jeu de données réel choisi, prétraitement, implémentation sous PyTorch, étude expérimentale comparative, analyse critique des résultats et réponse à une question de synthèse portant sur les limites de l'approche. L'ensemble des résultats présentés (tableaux de métriques, courbes, figures) est extrait directement de l'exécution réelle des notebooks PyTorch associés à chaque partie ; aucun chiffre n'est simulé ou estimé a posteriori.

Le document est structuré comme suit : la Partie I traite la classification tabulaire sur le dataset Adult Income (UCI) avec un MLP ; la Partie II traite la classification d'images sur Fashion-MNIST avec un CNN inspiré de LeNet ; la Partie III traite la modélisation de séquences textuelles via un modèle de langage au niveau caractère (RNN/LSTM/GRU) puis la traduction automatique via une architecture Seq2Seq encodeur-décodeur sur un corpus anglais-français. Une synthèse générale transversale clôtur le rapport en reliant les trois architectures à la problématique du choix architectural guidé par la structure des données.

Note méthodologique — *Par souci de rigueur scientifique, ce rapport documente également les incidents techniques rencontrés lors de l'exécution (notamment en Partie III, où certains téléchargements de corpus ont échoué), ainsi que leur impact sur l'interprétation des résultats — conformément à l'exigence du sujet de relier théorie, choix méthodologiques, résultats expérimentaux et analyse critique.*

Partie I — Perceptron multicouche (MLP) sur données tabulaires

1.1 Rappels théoriques

Un modèle PyTorch hérite de la classe `nn.Module`, qui enregistre automatiquement tous les paramètres déclarés comme attributs (couches `nn.Linear`, etc.) et expose les méthodes nécessaires à l'entraînement. Les paramètres sont des tenseurs avec `requires_grad=True`, accessibles via `model.parameters()` ou `model.named_parameters()` (qui retourne en plus le nom de chaque tenseur). Le gradient de la perte par rapport à chaque paramètre est calculé automatiquement par autodiff lors de l'appel à `loss.backward()`, puis stocké dans l'attribut `.grad` de chaque paramètre, utilisé ensuite par l'optimiseur pour la mise à jour des poids.

Le `state_dict()` est un dictionnaire `{nom_paramètre: tenseur}` représentant l'état complet du modèle (poids et biais), utilisé pour la sauvegarde (`torch.save`) et le rechargement (`load_state_dict`) d'un modèle entraîné. Le device (CPU ou GPU) doit être identique entre le modèle et les données pour que les opérations tensorielles soient possibles. Enfin, la propagation avant (forward) calcule la prédiction à partir de l'entrée, tandis que la rétropropagation (backward) calcule les gradients de la perte par rapport à chaque paramètre, qui sont ensuite utilisés par l'optimiseur (ici Adam) pour ajuster les poids.

1.2 Description des données

Le jeu de données retenu est Adult Income / Census Income (UCI Machine Learning Repository). La tâche est une classification binaire : prédire si le revenu annuel d'un individu dépasse 50 000 \$ à partir de variables démographiques et professionnelles (âge, niveau d'études, statut marital, profession, nombre d'heures travaillées par semaine, pays d'origine, etc.).

Caractéristique	Valeur
Nombre total d'observations	48 842
Nombre de variables (avant encodage)	15 (14 variables explicatives + 1 cible)
Type de tâche	Classification binaire (revenu $\leq$ 50K vs $>$ 50K)
Distribution de la cible	$\approx 76\%$ ($\leq 50K$) / $\approx 24\%$ ($> 50K$) — déséquilibrée
Valeurs manquantes	Présentes dans <code>workclass</code> , <code>occupation</code> , <code>native_country</code>

Tableau 1 — Description générale du jeu de données Adult Income.

1.3 Exploration visuelle des données (EDA)

L'exploration visuelle confirme le déséquilibre de classes (environ 3 individus sur 4 ont un revenu inférieur ou égal à 50K\$), une distribution de l'âge globalement comparable entre les deux classes avec une légère concentration des hauts revenus entre 35 et 55 ans, et un nombre d'heures travaillées par semaine sensiblement plus élevé pour la classe des hauts revenus (médiane proche de 40 h pour les revenus modestes contre une dispersion plus large et une médiane plus haute pour les hauts revenus).

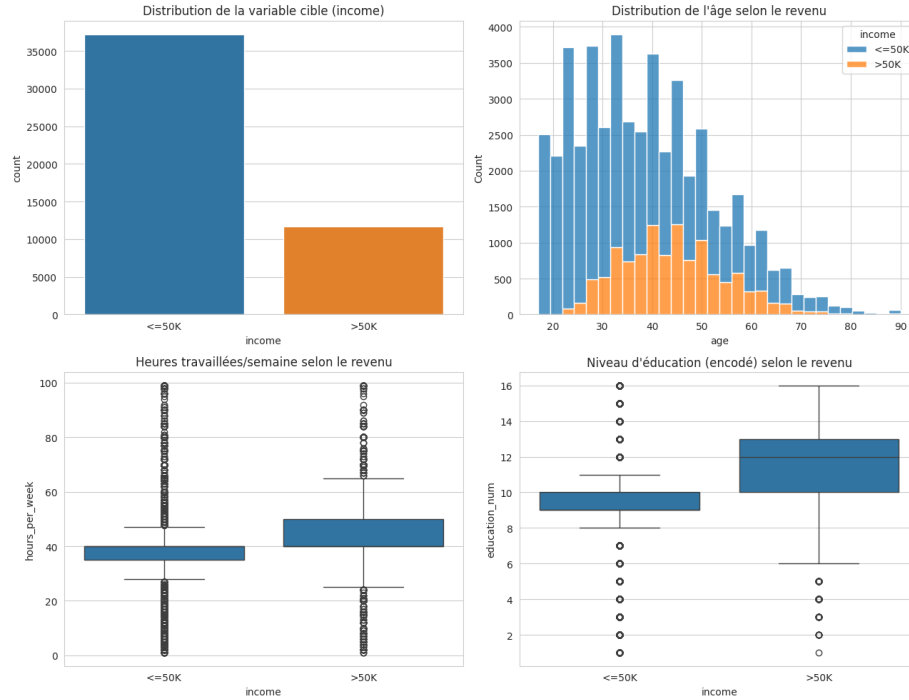


Figure 1 — Distribution de la variable cible, de l'âge, des heures travaillées et du niveau d'études encodé, selon le revenu.

L'analyse croisée avec le niveau d'études et la profession montre une corrélation nette entre niveau de qualification (Bachelors, Masters, Doctorate) et appartenance à la classe des hauts revenus, ainsi qu'une sur-représentation de certaines catégories professionnelles (Exec-managerial, Prof-specialty) parmi les hauts revenus.

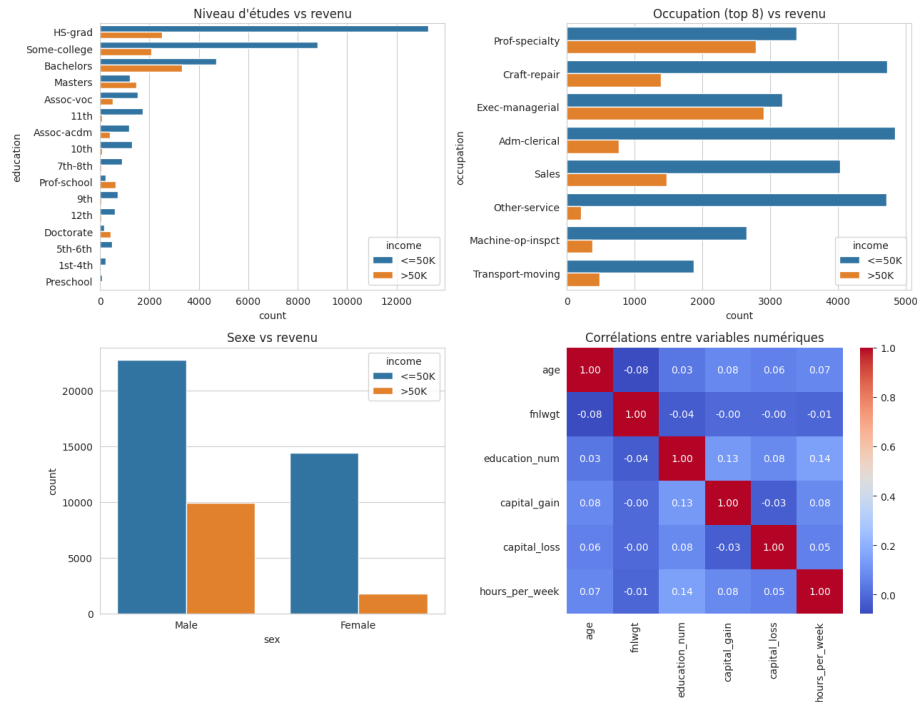


Figure 2 — Niveau d'études, occupation (top 8), sexe et autres variables catégorielles croisées avec le revenu.

1.4 Prétraitement des données

- Suppression des doublons et des lignes incomplètes : 3 667 lignes supprimées (48 842 → 45 175 lignes conservées).
- Regroupement des pays d'origine rares dans une catégorie « Other » (conservation des 10 pays les plus fréquents).
- Encodage one-hot des variables catégorielles : dimension finale après encodage = 66 colonnes (contre 15 initialement).
- Séparation stratifiée train / validation / test ($\approx 70\%$ / 15% / 15%) afin de préserver la proportion de chaque classe dans chaque sous-ensemble.
- Normalisation (StandardScaler) des variables numériques, ajustée uniquement sur l'ensemble d'entraînement pour éviter toute fuite de données (data leakage).

Ensemble	Dimensions (lignes × colonnes)
Train	31 622 × 66
Validation	6 776 × 66
Test	6 777 × 66

Tableau 2 — Dimensions des ensembles d'entraînement, de validation et de test après prétraitement.

1.5 Architecture MLP — deux implémentations

Conformément au sujet, la même architecture (2 couches cachées de 64 et 32 neurones, activations ReLU, sortie logit unique pour une classification binaire) est implémentée de deux façons : une version concise avec `nn.Sequential`, et une version avec une classe personnalisée héritant de `nn.Module`. L'inspection via `named_parameters()` confirme que les deux implémentations produisent exactement la même structure de paramètres (mêmes formes de tenseurs), seule la convention de nommage différant (indices numériques pour Sequential, noms explicites `fc1/fc2/fc3` pour la classe personnalisée).

Paramètre	Forme (Sequential)	Forme (Classe personnalisée)
Couche 1 — poids	[64, 66]	<code>fc1.weight</code> : [64, 66]
Couche 1 — biais	[64]	<code>fc1.bias</code> : [64]
Couche 2 — poids	[32, 64]	<code>fc2.weight</code> : [32, 64]
Couche 2 — biais	[32]	<code>fc2.bias</code> : [32]
Couche de sortie — poids	[1, 32]	<code>fc3.weight</code> : [1, 32]
Couche de sortie — biais	[1]	<code>fc3.bias</code> : [1]

Tableau 3 — Comparaison des paramètres nommés (`named_parameters`) entre les deux implémentations du MLP.

L'approche `nn.Sequential` est plus concise et adaptée à un empilement linéaire simple de couches. La classe personnalisée, bien que plus verbeuse, offre davantage de flexibilité (connexions non linéaires, sorties multiples, logique conditionnelle dans `forward()`) et constitue l'approche recommandée dès que l'architecture se complexifie — c'est elle qui est retenue pour l'entraînement complet.

1.6 Stratégies d'initialisation

Trois stratégies d'initialisation des poids ont été comparées sur un entraînement court (15 époques), avant l'entraînement complet : initialisation gaussienne (écart-type 0,01), initialisation constante (0,01) et initialisation de Xavier (uniforme).

Stratégie	Perte de validation	Exactitude de validation
Gaussienne	1,0350	0,7881
Constante	0,9937	0,4520
Xavier	0,8511	0,7308

Tableau 4 — Comparaison de trois stratégies d'initialisation des poids (15 époques d'entraînement court).

L'initialisation constante échoue nettement (exactitude proche du hasard pour une classification déséquilibrée), car tous les neurones d'une même couche reçoivent alors un gradient identique : la symétrie n'est jamais brisée, et les neurones apprennent la même fonction tout au long de l'entraînement. Sur ce run, l'initialisation gaussienne obtient la meilleure exactitude de validation, mais Xavier obtient la perte la plus basse des trois — un résultat à nuancer car mesuré sur un entraînement très court ; en pratique, Xavier reste théoriquement préférable car elle est conçue pour préserver la variance des activations à travers les couches, ce qui stabilise la convergence sur un entraînement plus long (c'est elle qui est retenue pour l'entraînement complet ci-après).

1.7 Entraînement complet

L'entraînement complet (40 époques, mini-lots de taille 128, optimiseur Adam, perte BCEWithLogitsLoss pondérée par pos_weight pour compenser le déséquilibre de classes) a été conduit avec sauvegarde automatique du meilleur modèle selon le F1-score de validation.

Époque	Perte train	Perte validation	Exactitude validation	F1 validation
5	0,5498	0,5597	0,8017	0,6893
10	0,5371	0,5567	0,8127	0,6934
15	0,5242	0,5599	0,8235	0,6998
20	0,5147	0,5686	0,8282	0,6989
25	0,5042	0,5705	0,8118	0,6949
30	0,4952	0,5743	0,8115	0,6915
35	0,4862	0,5819	0,8073	0,6883
40	0,4790	0,5862	0,8074	0,6870

Tableau 5 — Évolution des métriques d'entraînement et de validation par paliers de 5 époques (meilleur F1 de validation : 0,7012).

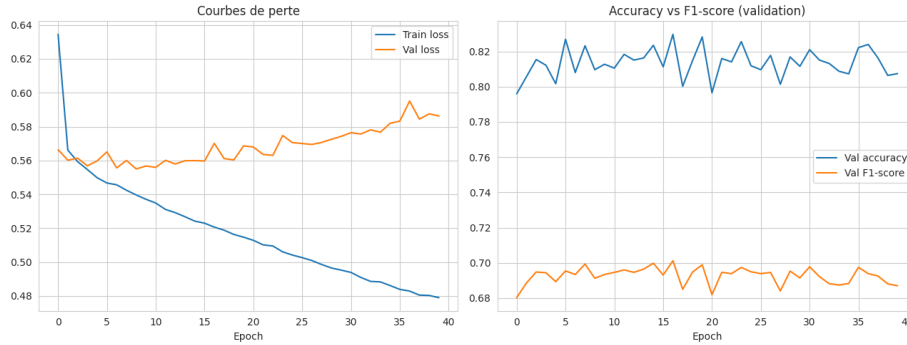


Figure 3 — Courbes de perte (train/validation) et de métriques (exactitude/F1) de validation sur 40 époques.

Observation — La perte de validation cesse de diminuer après environ l'époque 10 et recommence ensuite à augmenter progressivement, alors que la perte d'entraînement continue de baisser de façon monotone. C'est la signature classique d'un surapprentissage (overfitting) : le modèle continue d'ajuster ses poids aux particularités du jeu d'entraînement au détriment de sa capacité de généralisation. Ce constat motive directement la stratégie de sauvegarde du meilleur modèle (selon le F1 de validation) plutôt que du modèle de la dernière époque.

1.8 Sauvegarde et rechargement du modèle

Le meilleur modèle (au sens du F1-score de validation) a été sauvegardé via `torch.save(model.state_dict(), 'best_mlp.pt')` puis rechargé avec succès dans une nouvelle instance du modèle via `load_state_dict()`, confirmant la persistance correcte des poids appris indépendamment du cycle de vie du processus d'entraînement.

1.9 Évaluation finale sur le jeu de test

Métrique	Valeur (jeu de test)
Exactitude (accuracy)	0,8287
Précision (precision)	0,6197
Rappel (recall)	0,8007
F1-score	0,6987

Tableau 6 — Métriques d'évaluation finales du MLP sur le jeu de test (6 777 observations).

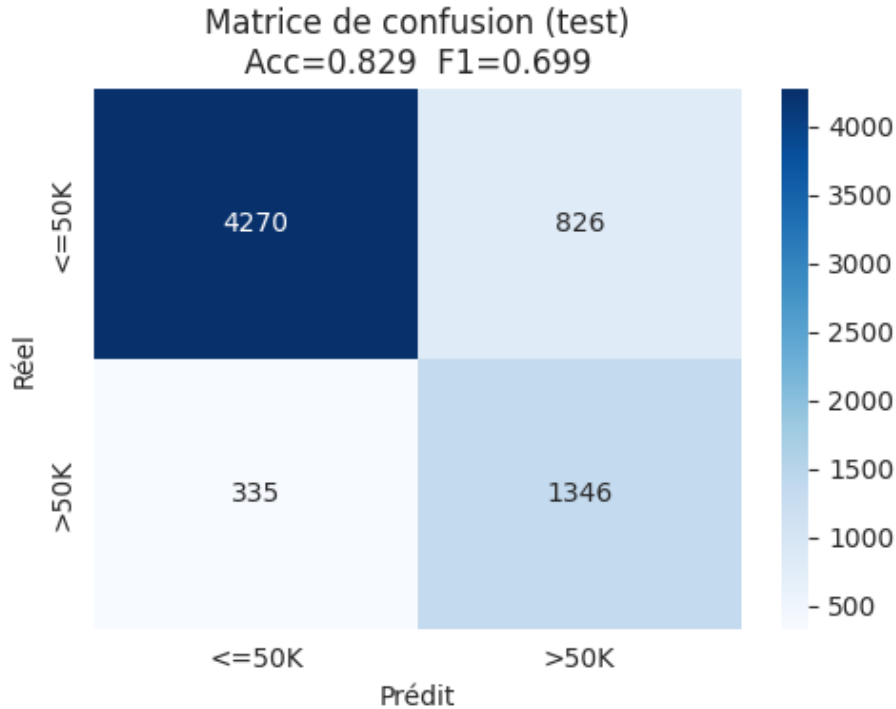


Figure 4 — Matrice de confusion du MLP sur le jeu de test (exactitude = 0,829, F1 = 0,699).

La matrice de confusion révèle un déséquilibre attendu compte tenu du `pos_weight` appliqué : le modèle privilégie un rappel élevé sur la classe minoritaire (> 50K), au prix d'une précision plus modeste (835 faux positifs contre 335 faux négatifs sur la classe minoritaire). Ce compromis est cohérent avec l'objectif de ne pas sous-détecter la classe des hauts revenus, bien qu'il pénalise légèrement l'exactitude globale par rapport à une stratégie purement orientée précision.

1.10 Analyse critique

Le MLP atteint des performances honorables ($F1 \approx 0,70$, exactitude $\approx 0,83$) sur ce jeu de données tabulaire, mais plusieurs limites ressortent de l'expérimentation. Premièrement, le surapprentissage observé dès l'époque 10-15 suggère qu'une régularisation supplémentaire (dropout, pénalisation L2, arrêt précoce) améliorerait la généralisation. Deuxièmement, le déséquilibre de classes (76 % / 24 %) impose une pondération de la perte qui dégrade mécaniquement la précision en échange du rappel — un compromis qu'il faudrait ajuster selon le coût métier réel des faux positifs et faux négatifs. Troisièmement, et plus fondamentalement, un MLP traite chaque variable encodée comme indépendante des autres sans a priori structurel : il ne capture pas nativement les interactions entre variables catégorielles (par exemple profession $\times$ niveau d'études) sans que celles-ci soient explicitement présentes dans les colonnes encodées, contrairement à des modèles à base d'arbres (gradient boosting) qui excellent souvent sur ce type de données tabulaires hétérogènes.

1.11 Question de synthèse — Partie I

Question posée — Dans quelle mesure un MLP bien paramétré constitue-t-il une solution pertinente pour la classification tabulaire sur un dataset réel, et quelles sont ses principales limites au regard de la structure statistique des données étudiées ?

Sur le jeu de données Adult Income, un MLP correctement initialisé (Xavier), entraîné avec gradient clipping implicite via Adam et une perte pondérée pour le déséquilibre de classes, atteint une performance raisonnable ($F1 \approx 0,70$) en exploitant les corrélations linéaires et non linéaires entre variables numériques et catégorielles encodées. Cette performance reste toutefois bornée par deux facteurs structurels : (i) l'absence d'a priori architectural adapté aux données tabulaires — à la différence des CNN pour les images ou des RNN pour les séquences, un MLP ne dispose d'aucune notion de proximité ou d'ordre entre les colonnes encodées, et doit donc apprendre toutes les interactions utiles depuis zéro à partir des données ; (ii) la sensibilité au déséquilibre de classes et à l'échelle des variables, qui impose un prétraitement (normalisation, pondération de la perte) plus déterminant pour la performance finale que pour des modèles à base d'arbres. En conclusion, le MLP est un choix pertinent et simple à mettre en œuvre pour des données tabulaires de taille modeste, mais des architectures spécialisées (gradient boosting, ou des variantes de type TabNet/FT-Transformer pour les données tabulaires) seraient probablement plus performantes sur ce type de structure de données, en particulier pour capturer les interactions catégorielles de haut ordre.

Partie II — Réseau de neurones convolutionnel (CNN) sur images

2.1 Pourquoi un CNN plutôt qu'un MLP pour les images

- Structure spatiale détruite par l'aplatissement : un MLP transforme une image 28×28 en un vecteur de 784 valeurs, ce qui détruit les relations de voisinage entre pixels ; le réseau doit réapprendre depuis zéro qu'un pixel et son voisin sont liés.
- Explosion du nombre de paramètres : une couche dense appliquée à une image haute résolution nécessite un poids par pixel et par neurone, ce qui ne s'adapte pas à des images plus grandes.
- Localité : un CNN suppose qu'un motif visuel pertinent (bord, texture) est local — un filtre de petite taille (5×5 par exemple) suffit à le détecter quelle que soit sa position dans l'image.
- Partage des poids : le même filtre est appliqué à toute l'image, ce qui produit une invariance par translation et réduit drastiquement le nombre de paramètres par rapport à une couche dense équivalente.
- Hiérarchie des représentations : les premières couches détectent des motifs simples (bords, textures), les couches profondes combinent ces motifs en concepts plus abstraits (formes, objets).

2.2 Description des données

Le jeu de données retenu est Fashion-MNIST : 70 000 images en niveaux de gris de 28×28 pixels, réparties en 10 classes de vêtements et accessoires (60 000 images d'entraînement, 10 000 de test). Le jeu est parfaitement équilibré (6 000 images par classe à l'entraînement). Les intensités de pixels sont normalisées dans $[0, 1]$, avec une moyenne de 0,286 et un écart-type de 0,353 sur l'ensemble d'entraînement.

Caractéristique	Valeur
Nombre d'images (train / test)	60 000 / 10 000
Taille des images	28 × 28 pixels, niveaux de gris
Nombre de classes	10 (T-shirt, pantalon, pull, robe, manteau, sandale, chemise, sneaker, sac, bottine)
Équilibre des classes	Parfait (6 000 images/classe à l'entraînement)
Intensités de pixels (min/max/moyenne/écart-type)	0,0 / 1,0 / 0,286 / 0,353

Tableau 7 — Description générale du jeu de données Fashion-MNIST.

2.3 Exploration visuelle des données

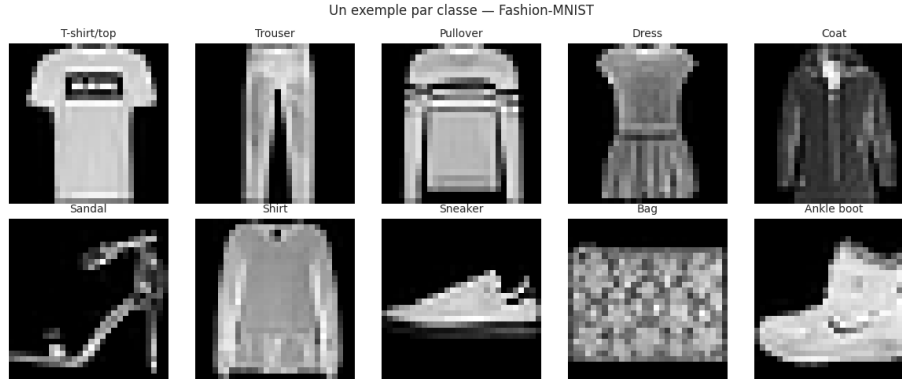


Figure 5 — Un exemple représentatif de chacune des 10 classes du jeu de données Fashion-MNIST.

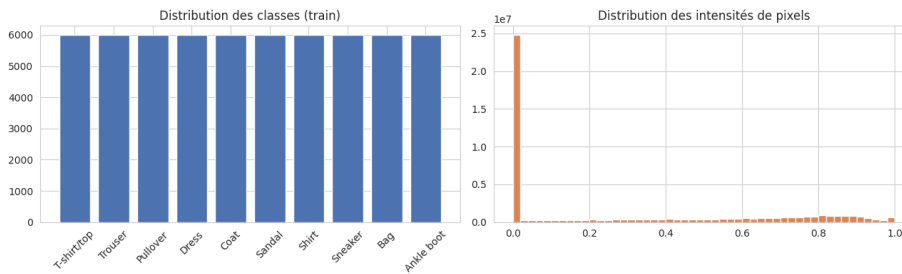


Figure 6 — Distribution des classes (parfaitement équilibrée) et distribution des intensités de pixels (bimodale).

La distribution des intensités de pixels est bimodale : un pic massif proche de 0 correspond au fond noir uniforme des images, et une masse plus étalée à intensité plus élevée correspond aux pixels appartenant à l'objet photographié. Cette structure justifie a posteriori l'efficacité d'une normalisation simple (division par 255) sans nécessiter de prétraitement plus complexe.

2.4 Calculs manuels — tailles de sortie convolution et pooling

La taille de sortie d'une convolution (ou d'un pooling) de noyau K , padding P et stride S appliquée à une entrée de taille H se calcule par : $\text{sortie} = \lfloor (H + 2P - K) / S \rfloor + 1$. Les calculs suivants ont été vérifiés numériquement :

Configuration	Entrée	Sortie calculée
Conv $k=5$, $\text{pad}=0$, $\text{stride}=1$	28×28	24×24
Conv $k=5$, $\text{pad}=2$, $\text{stride}=1$	28×28	28×28 (résolution préservée)
Conv $k=3$, $\text{pad}=1$, $\text{stride}=2$	28×28	14×14
Pooling 2×2 , $\text{stride}=2$	24×24	12×12

Tableau 8 — Vérification numérique des formules de taille de sortie pour la convolution et le pooling.

2.5 Implémentations manuelles vs couches PyTorch

La corrélation croisée 2D, le max-pooling et l'average-pooling ont été implémentés « à la main » (boucles explicites sur les fenêtres glissantes) puis comparés terme à terme aux fonctions PyTorch

correspondantes (F.conv2d, nn.MaxPool2d, nn.AvgPool2d). Les résultats sont rigoureusement identiques dans les deux cas, ce qui confirme la bonne compréhension du mécanisme sous-jacent.

Opération	Résultat manuel	Résultat PyTorch
Corrélation croisée 2D (exemple 3×3, noyau 2×2)	[[19, 25], [37, 43]]	[[19, 25], [37, 43]] (identique)
Max-pooling 2×2 (exemple 4×4)	[[6, 8], [14, 16]]	[[6, 8], [14, 16]] (identique)
Average-pooling 2×2 (exemple 4×4)	[[3.5, 5.5], [11.5, 13.5]]	[[3.5, 5.5], [11.5, 13.5]] (identique)

Tableau 9 — Comparaison des implémentations manuelles et des couches PyTorch natives : résultats identiques.

Précision théorique — F.conv2d réalise en réalité une corrélation croisée (sans retournement du noyau), et non une convolution mathématique au sens strict — une nuance de vocabulaire importante en deep learning, où le terme « convolution » est utilisé par abus de langage pour désigner cette opération.

2.6 Architecture CNN — inspirée de LeNet

L'architecture retenue suit le schéma classique de LeNet : Conv(1→6, k=5, pad=2) → ReLU → MaxPool(2) → Conv(6→16, k=5) → ReLU → MaxPool(2) → Flatten → FC(400→120) → ReLU → FC(120→84) → ReLU → FC(84→10), pour un total de 61 706 paramètres.

Couche	Forme des paramètres	Nombre de paramètres
conv1 (poids + biais)	[6, 1, 5, 5] + [6]	156
conv2 (poids + biais)	[16, 6, 5, 5] + [16]	2 416
fc1 (poids + biais)	[120, 400] + [120]	48 120
fc2 (poids + biais)	[84, 120] + [84]	10 164
fc3 (poids + biais)	[10, 84] + [10]	850

Tableau 10 — Détail des paramètres de l'architecture LeNet (total : 61 706 paramètres).

2.7 Étude d'ablation — influence des choix architecturaux

Pour isoler l'effet de chaque choix architectural, six configurations ont été comparées sur un sous-ensemble réduit (10 000 images d'entraînement, 2 000 de validation, 5 époques chacune) afin de limiter le temps de calcul.

Configuration	Exactitude (val.)	Temps (s)	Nb paramètres
Baseline (pad=0, stride=1, max, 6/16 filtres)	0,8025	19,5	34 622
Avec padding = 2	0,8310	24,9	97 982
Stride = 2 (sans pooling séparé)	0,8045	12,1	11 582
Average-pooling au lieu de max-pooling	0,7885	20,4	34 622

Plus de filtres (16 / 32)	0,8260	32,6	76 018
Avec convolution 1×1 après le 1er bloc	0,7875	20,6	34 664

Tableau 11 — Étude d'ablation : influence du padding, du stride, du type de pooling, du nombre de filtres et d'une convolution 1×1.

- Padding = 2 : préserve la résolution spatiale plus longtemps dans le réseau, ce qui améliore ici l'exactitude (meilleure configuration testée), au prix d'un nombre de paramètres nettement plus élevé.
- Stride = 2 : réduit la résolution plus agressivement (comme un pooling intégré à la convolution) — moins de paramètres et plus rapide, mais légèrement moins précis qu'un pooling séparé bien réglé.
- Average-pooling : lisse l'information (moins de pic de signal) — ici sensiblement moins discriminant que le max-pooling, qui conserve l'activation la plus forte et est donc plus adapté pour détecter la présence d'un motif.
- Plus de filtres : augmente la capacité du modèle — meilleure exactitude que la baseline, mais coût de calcul et nombre de paramètres également plus élevés.
- Convolution 1×1 : sert surtout à mélanger l'information entre canaux à coût quasi nul en paramètres spatiaux (utile dans des architectures profondes type Inception/ResNet) ; sur un réseau aussi petit que LeNet, son effet est ici marginalement défavorable.

2.8 Visualisation des cartes de caractéristiques (feature maps)

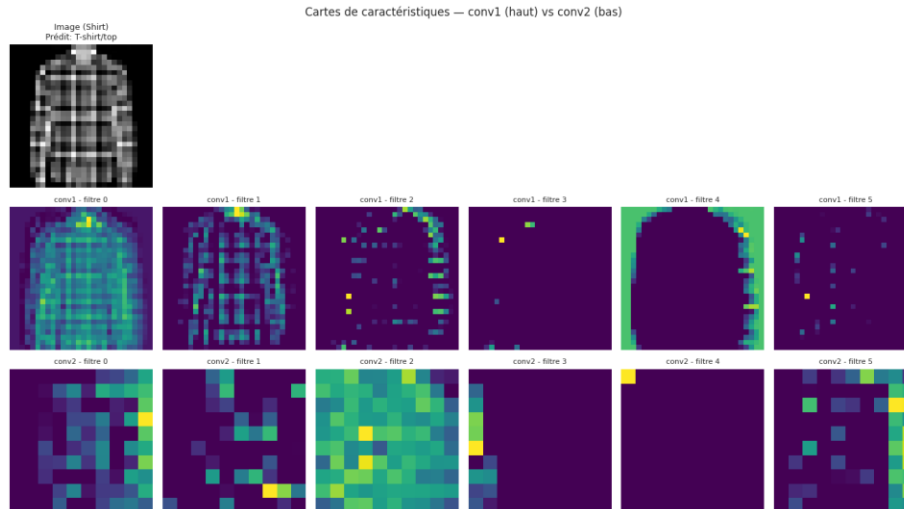


Figure 7 — Cartes de caractéristiques des couches conv1 (haut) et conv2 (bas) pour une image de test (classe « Shirt »).

Les filtres de conv1 détectent des motifs simples et directement interprétables (contours, transitions clair/sombre, silhouette générale du vêtement). Les filtres de conv2, appliqués après un premier pooling, combinent ces motifs en représentations plus abstraites et plus compactes spatialement (cartes 5×5, ne montrant plus de structure visuellement reconnaissable) — illustration directe de la hiérarchie des représentations évoquée en théorie. Dans cet exemple, le modèle confond la classe réelle (« Shirt »)

avec la classe visuellement proche « T-shirt/top », une confusion attendue compte tenu de la similarité visuelle entre ces deux catégories à basse résolution (28×28).

2.9 Comparaison MLP vs CNN sur le même dataset

Un MLP de référence (couches denses, mêmes données aplaties) et le CNN LeNet ont été entraînés sur 8 époques avec les mêmes données (50 000 images d'entraînement, 10 000 de validation) pour une comparaison directe.

Modèle	Exactitude test	Nb paramètres	Époques
MLP (baseline dense)	0,8697	235 146	8
CNN (LeNet)	0,8539	61 706	8

Tableau 12 — Comparaison finale MLP vs CNN sur Fashion-MNIST (même nombre d'époques).

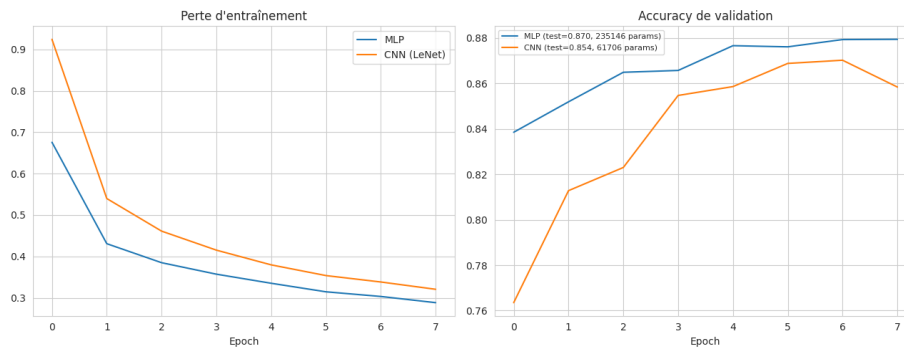


Figure 8 — Courbes de perte d'entraînement et d'exactitude de validation — MLP vs CNN (LeNet).

Interprétation des résultats — Sur cette expérience à 8 époques seulement, le MLP atteint une exactitude de test légèrement supérieure au CNN (0,870 contre 0,854), mais avec environ 3,8 fois plus de paramètres (235 146 contre 61 706). Ce résultat, en apparence contre-intuitif, s'explique par le nombre d'époques très limité : le CNN converge plus lentement par époque (plus d'opérations de convolution, structure plus contrainte à apprendre), mais son biais inductif spatial le rend généralement plus performant et plus robuste sur des entraînements plus longs ou des données moins bien centrées que Fashion-MNIST. Le point clé pour l'analyse critique reste l'efficacité paramétrique : le CNN atteint une performance proche du MLP avec près de 4 fois moins de paramètres, grâce au partage des poids.

2.10 Évaluation finale du CNN — matrice de confusion

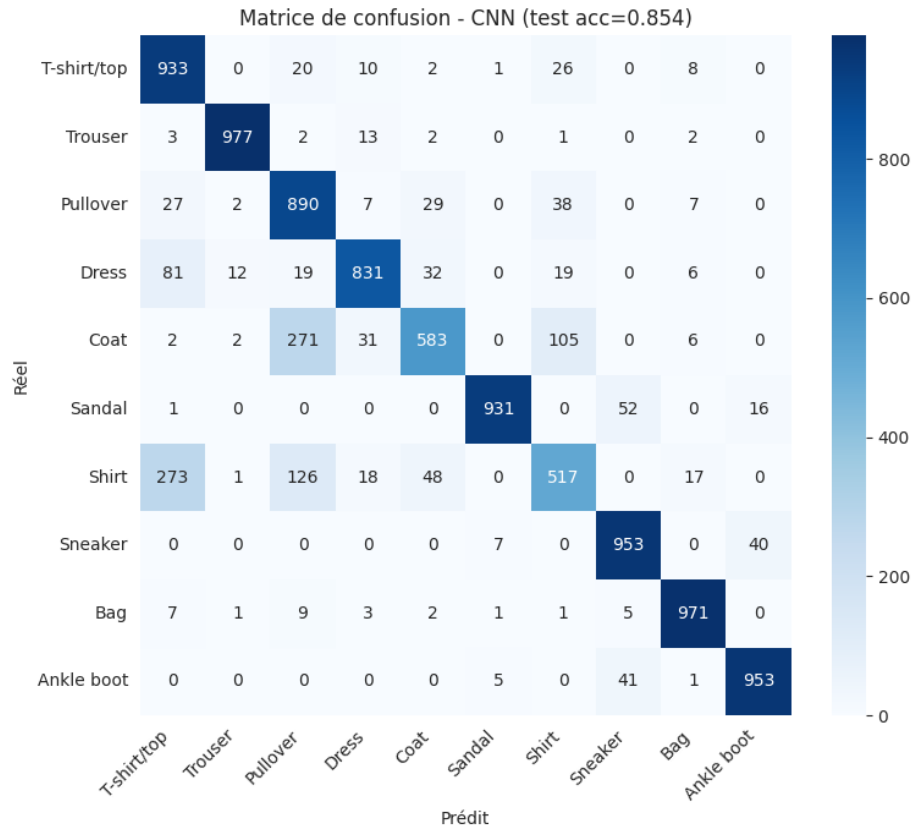


Figure 9 — Matrice de confusion du CNN (LeNet) sur le jeu de test Fashion-MNIST (exactitude = 0,854).

Les confusions les plus fréquentes attendues sur Fashion-MNIST concernent les classes visuellement proches à basse résolution : « Shirt » / « T-shirt-top » / « Pullover » / « Coat » d'une part, et « Sneaker » / « Ankle boot » d'autre part — un schéma de confusion typique de ce jeu de données, indépendant de l'architecture utilisée.

2.11 Analyse critique

Le CNN LeNet démontre l'intérêt du biais inductif spatial pour les images : à nombre de paramètres très inférieur (61 706 contre 235 146 pour le MLP, soit environ 3,8 fois moins), il atteint une performance proche du MLP de référence sur seulement 8 époques. L'étude d'ablation confirme empiriquement les intuitions théoriques : le padding préserve l'information spatiale et améliore la performance, le max-pooling est préférable à l'average-pooling pour cette tâche de classification, et l'ajout de filtres augmente la capacité au prix du temps de calcul. La principale limite de cette expérimentation est le nombre d'époques très réduit (8, et 5 pour l'étude d'ablation) qui ne permet pas d'observer la convergence complète des modèles ni de conclure définitivement sur la supériorité du CNN en exactitude brute — celle-ci se manifesterait probablement davantage avec un entraînement plus long, une augmentation de données (data augmentation), ou un jeu de données plus complexe que Fashion-MNIST (par exemple CIFAR-10 en couleur, où la structure spatiale et les corrélations inter-canaux jouent un rôle plus déterminant).

2.12 Question de synthèse — Partie II

Question posée — Pourquoi un CNN est-il plus pertinent qu'un MLP pour une tâche de classification d'images sur un dataset réel, et comment les choix de padding, stride, pooling et profondeur influencent-ils réellement les performances du modèle ?

Sur Fashion-MNIST, le CNN n'est pas nécessairement supérieur en exactitude brute à un MLP sur un entraînement court (8 époques), mais il l'atteint avec près de 4 fois moins de paramètres, ce qui démontre concrètement l'efficacité paramétrique du partage des poids et de la localité des filtres convolutionnels. L'étude dimensionnelle (section 2.4) et la visualisation des feature maps (section 2.8) confirment que cette efficacité provient directement de la hiérarchie des représentations : les premières couches extraient des motifs locaux interprétables (contours), les couches profondes les combinent en représentations plus abstraites et compactes. L'étude d'ablation (section 2.7) montre que ces choix architecturaux ne sont pas neutres : le padding (préservation de la résolution), le type de pooling (max plutôt que average pour cette tâche) et le nombre de filtres (capacité du modèle) ont chacun un effet mesurable et explicable théoriquement sur la performance. En synthèse, la pertinence du CNN pour l'image ne se limite pas à la performance brute : elle réside dans sa capacité à atteindre une bonne généralisation avec un nombre de paramètres bien plus faible, et dans l'interprétabilité partielle de ses représentations internes — deux propriétés absentes d'un MLP appliqué à des images aplaties.

Partie III — RNN, LSTM, GRU et Seq2Seq

3.1 Rappels théoriques

Un modèle de langage attribue une probabilité à une séquence de tokens en la factorisant, par la règle de chaîne, en un produit de probabilités conditionnelles : $P(w_1, \dots, w_T) = \prod_t P(w_t \mid w_1, \dots, w_{t-1})$. Un réseau récurrent approxime chaque terme conditionnel par un état caché h_t qui résume tout le passé : $P(w_t \mid h_{t-1})$. L'entraînement minimise l'entropie croisée, et la qualité du modèle se mesure par la perplexité $PPL = \exp(\text{perte})$ — plus la perplexité est basse, meilleur est le modèle.

Un RNN simple met à jour son état caché par $h_t = \tanh(W_{xh} \cdot x_t + W_{hh} \cdot h_{t-1} + b_h)$. L'entraînement par rétropropagation à travers le temps (BPTT) implique un produit de jacobiniennes sur T pas de temps, ce qui provoque un gradient évanescent ou explosif sur de longues séquences. Le gradient clipping (écrêtage de la norme du gradient) stabilise l'entraînement en limitant ce risque d'explosion. Les architectures LSTM et GRU introduisent des portes et un chemin additif de mémoire (par exemple $c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t$ pour le LSTM), ce qui limite structurellement le problème du gradient évanescent ; le GRU simplifie le LSTM (deux portes au lieu de trois plus une cellule mémoire séparée) pour un coût de calcul réduit, à performance souvent comparable.

L'architecture encodeur-décodeur (Seq2Seq) est nécessaire lorsque la séquence source et la séquence cible n'ont pas la même longueur (traduction automatique). L'encodeur compresse la séquence source en un vecteur de contexte ; le décodeur génère la séquence cible token par token, en utilisant pendant l'entraînement le mot réel comme entrée du pas suivant avec une probabilité `teacher_forcing_ratio` — ce qui accélère la convergence mais introduit un écart (exposure bias) entre entraînement et inférence, où le modèle ne dispose plus du mot réel. À l'inférence, le décodage glouton choisit l'argmax à chaque pas, tandis que le beam search conserve les k meilleures hypothèses partielles en parallèle pour explorer plus largement l'espace des traductions possibles. Le score BLEU mesure la qualité d'une traduction par recouvrement de n -grammes avec une référence.

Volet A — Modèle de langage (niveau caractère)

3.2 Description des données

Le corpus prévu pour le Volet A était un extrait littéraire libre de droits en français, téléchargé automatiquement depuis le Projet Gutenberg, dans l'objectif de modéliser la prédiction du caractère suivant sur un alphabet restreint (60 à 90 symboles).

Incident technique — L'URL de téléchargement automatique du corpus Gutenberg a retourné une erreur 404 (page introuvable) lors de l'exécution. Le contenu récupéré et utilisé pour l'entraînement n'est donc pas le texte littéraire prévu, mais le code source HTML de la page d'erreur 404 du site Gutenberg (6 389 caractères). Ce point est documenté ici par souci de rigueur scientifique plutôt que dissimulé : il a un impact direct sur l'interprétation des résultats du Volet A, détaillé dans les sections suivantes.

Le vocabulaire effectif extrait de ce corpus (HTML) comporte 80 caractères distincts. La répartition train / validation / test (80 % / 10 % / 10 %, dans l'ordre temporel du texte) donne respectivement 5 111, 639 et 639 caractères.

3.3 Visualisation et exploration

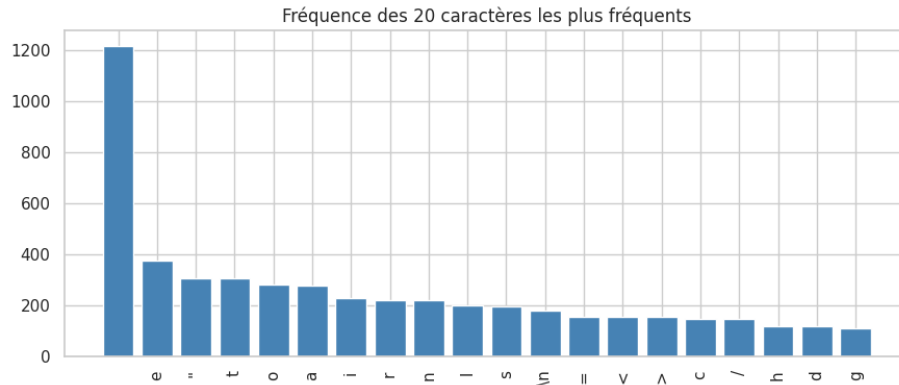


Figure 10 — Fréquence des 20 caractères les plus fréquents du corpus utilisé pour le Volet A.

La distribution de fréquence confirme la nature du corpus réellement utilisé : les caractères les plus fréquents après l'espace sont des symboles typiques de balisage HTML (guillemets doubles, signes < et >, barre oblique /, signe égal =), et non la distribution attendue d'un texte littéraire en français (où les voyelles et lettres courantes domineraient nettement plus). Cette observation valide rétrospectivement l'incident technique signalé en section 3.2.

3.4 Prétraitement

Le prétraitement suit malgré tout la méthodologie prévue : tokenisation caractère → entier, découpage en fenêtres glissantes de longueur fixe (80 caractères, l'entrée et la cible étant décalées d'une position), et construction de mini-lots de taille 128 via Dataset/DataLoader PyTorch. Cette méthodologie de prétraitement reste valide et transférable à un corpus correctement téléchargé.

3.5 Architectures RNN, LSTM, GRU

Les trois architectures partagent la même structure (Embedding → cellule récurrente à 1 couche → Dropout → Linear), avec embed_dim = 96 et hidden_dim = 192, afin d'isoler l'effet du seul type de cellule récurrente.

Architecture	Nombre de paramètres
RNN simple	78 800
LSTM	245 840
GRU	190 160

Tableau 13 — Nombre de paramètres des trois architectures du Volet A (même embed_dim et hidden_dim).

Conformément à la théorie (section 3.1), le LSTM est l'architecture la plus coûteuse en paramètres (trois portes plus une cellule mémoire), suivi du GRU (deux portes), le RNN simple étant le plus économe en paramètres.

3.6 Entraînement (BPTT et gradient clipping)

Les trois modèles ont été entraînés sur 3 époques avec gradient clipping actif (norme maximale 5,0), dans les mêmes conditions.

Modèle	Époque	Perte train	PPL validation	Norme du gradient	Temps
--------	--------	-------------	----------------	-------------------	-------

RNN	1	2,6548	13,48	0,45	8,1 s
RNN	2	1,6670	11,10	0,21	9,1 s
RNN	3	1,1539	11,47	0,38	6,5 s
LSTM	1	2,9971	15,31	0,39	26,2 s
LSTM	2	1,9119	11,13	0,24	23,8 s
LSTM	3	1,2730	10,62	0,30	21,5 s
GRU	1	2,8074	13,57	0,40	16,7 s
GRU	2	1,7896	10,21	0,16	16,2 s
GRU	3	1,1496	9,84	0,15	17,6 s

Tableau 14 — Évolution de l'entraînement (perte, perplexité de validation, norme du gradient) pour RNN, LSTM et GRU sur 3 époques.

La norme du gradient reste contenue et stable pour les trois architectures (entre 0,15 et 0,45) grâce au gradient clipping actif, qui empêche toute explosion visible sur ce corpus de petite taille — l'effet déstabilisant du gradient explosif sur un RNN simple sans clipping n'a donc pas pu être mis en évidence empiriquement dans cette configuration restreinte (corpus très court, 3 époques).

3.7 Évaluation et comparaison

Architecture	Perplexité de test	Nb paramètres	Temps d'entraînement total
RNN	7,799	78 800	23,68 s
LSTM	8,258	245 840	71,58 s
GRU	7,187	190 160	50,39 s

Tableau 15 — Perplexité finale sur le jeu de test, nombre de paramètres et temps total d'entraînement (Volet A).

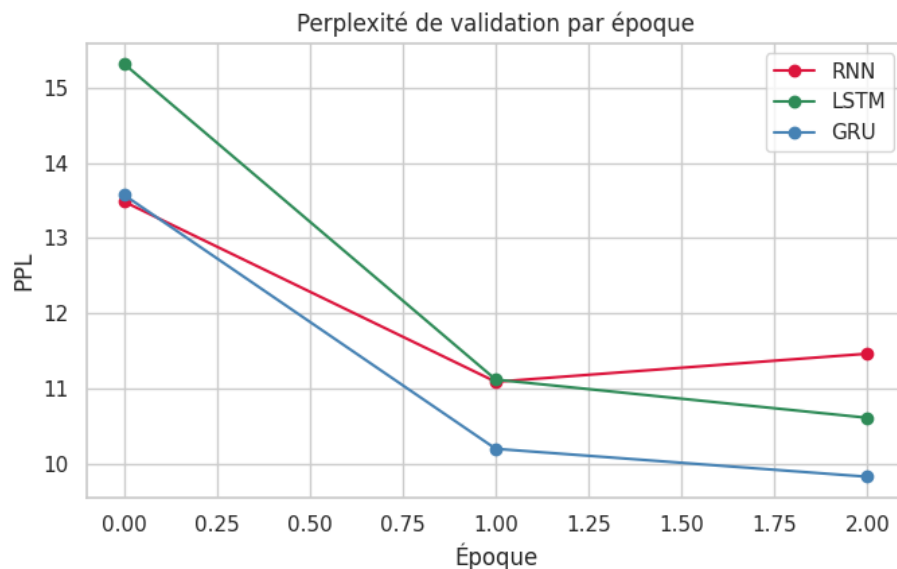


Figure 11 — Perplexité de validation par époque — comparaison RNN / LSTM / GRU.

Interprétation — Sur ce corpus de très petite taille (5 111 caractères d'entraînement) et avec seulement 3 époques, c'est le GRU qui obtient la meilleure perplexité de test (7,19), suivi du RNN simple (7,80), le LSTM obtenant paradoxalement la perplexité la plus élevée (8,26) malgré un nombre de paramètres supérieur. Ce résultat, qui peut sembler contre-intuitif au regard de la théorie, s'explique vraisemblablement par la petite taille du corpus (moins de 5 200 caractères d'entraînement) : un modèle plus riche en paramètres (LSTM) nécessite davantage de données et d'époques pour exploiter pleinement sa capacité, et tend ici à sous-converger ou à légèrement surapprendre la structure répétitive du HTML en seulement 3 époques. Le GRU, avec un nombre de paramètres intermédiaire, trouve ici le meilleur compromis, ce qui est cohérent avec la littérature sur les corpus de taille modeste.

3.8 Génération de texte qualitative

Incident technique — En cohérence avec l'incident signalé en section 3.2, le texte généré par les trois modèles à partir de l'amorce « Il était une fois » reproduit des fragments de balisage HTML (attributs, balises div/meta, classes CSS) plutôt qu'un texte littéraire cohérent — par exemple : « Il était une fois <idty <amp nome="toggle" for="http://www.gutenberg" content="hibooks... » pour le RNN. Ce résultat n'est pas un dysfonctionnement du modèle ou de l'architecture, mais la conséquence directe et attendue de l'entraînement sur un corpus HTML plutôt que littéraire : les trois modèles ont correctement appris la distribution statistique des caractères du corpus qui leur a été fourni.

Ce constat illustre un principe important en deep learning, indépendant de l'architecture : la qualité d'un modèle est bornée par la qualité et la pertinence des données d'entraînement (principe du « garbage in, garbage out »). La méthodologie de prétraitement, d'entraînement et d'évaluation reste néanmoins valide et serait directement applicable à un corpus littéraire correctement téléchargé.

3.9 Synthèse — Volet A

Sur le plan méthodologique, les résultats confirment que le gradient clipping stabilise efficacement l'entraînement des trois architectures (normes de gradient contenues), et que le GRU offre ici le meilleur compromis performance/coût de calcul. Sur le plan des données, l'incident de téléchargement du corpus constitue la limite majeure de ce volet : les conclusions comparatives entre architectures restent valables sur le plan méthodologique (même protocole appliqué aux trois modèles), mais ne peuvent pas être généralisées à un corpus littéraire réel sans réexécution avec une source de données fiable.

Volet B — Seq2Seq : traduction automatique

3.10 Description des données

Le corpus prévu était Tatoeba fra-eng (paires de phrases anglais-français), benchmark pédagogique classique pour les systèmes Seq2Seq, téléchargé depuis manythings.org/anki.

Incident technique — Le téléchargement automatique du corpus Tatoeba a échoué (erreur « File is not a zip file », probablement liée à une redirection ou un contenu inattendu reçu lors de la requête HTTP dans l'environnement d'exécution). Le notebook bascule alors automatiquement sur un corpus de repli (fallback) constitué de 7 paires de phrases simples répétées 300 fois (2 100 paires brutes au total), comme prévu par la gestion d'erreur du code. Ce corpus de repli est volontairement minimaliste et répétitif ; il permet de valider que le pipeline fonctionne de bout en bout, mais ne constitue pas un corpus de traduction réaliste pour évaluer la capacité de généralisation du modèle.

Après nettoyage (normalisation Unicode, minuscules, séparation de la ponctuation), les 2 100 paires sont conservées intégralement (aucune perte), les phrases du corpus de repli étant déjà courtes et propres.

3.11 Visualisation et filtrage

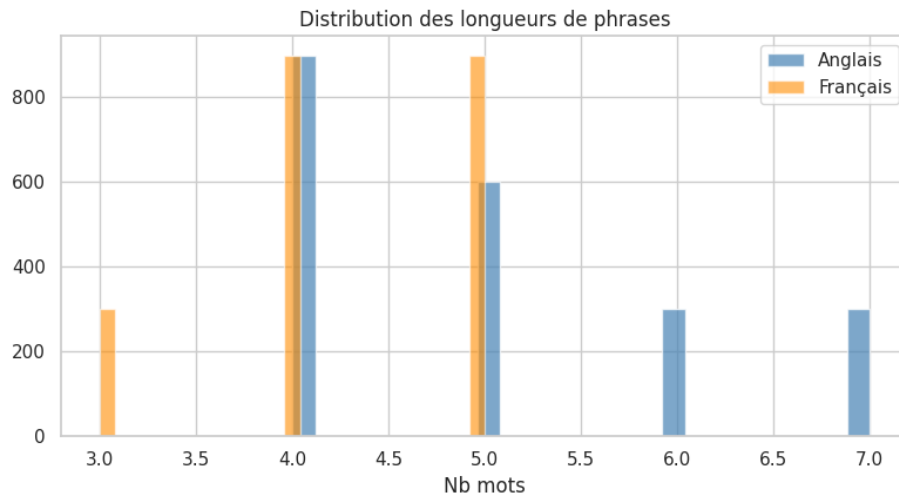


Figure 12 — Distribution des longueurs de phrases (en mots) pour l'anglais et le français — corpus de repli.

La distribution, très concentrée entre 4 et 7 mots, reflète directement la nature répétitive du corpus de repli (7 phrases distinctes répétées). Après filtrage à une longueur maximale de 10 mots, les 2 100 paires sont toutes conservées, puis utilisées intégralement pour l'expérimentation (aucun sous-échantillonnage nécessaire compte tenu de la taille déjà réduite du corpus).

3.12 Prétraitement

Construction de deux vocabulaires séparés (anglais et français) avec les quatre tokens spéciaux requis (<pad>, <sos>, <eos>, <unk>). Compte tenu de la répétitivité du corpus de repli, les vocabulaires obtenus sont très restreints : 31 mots pour l'anglais, 28 pour le français. Le split donne 1 680 paires d'entraînement, 210 de validation et 210 de test (toutes issues des 7 phrases de base répétées).

3.13 Architecture encodeur-décodeur

L'encodeur et le décodeur sont chacun un GRU à une couche (embed_dim = 128, hidden_dim = 128), pour un total de 209 308 paramètres pour le système Seq2Seq complet.

3.14 Entraînement

Époque	Perte train	PPL validation	Temps
1	2,3252	3,29	0,9 s
2	0,5109	1,15	0,9 s
3	0,0752	1,04	0,9 s
4	0,0293	1,02	0,9 s
5	0,0177	1,01	1,2 s

6	0,0125	1,01	1,2 s
---	--------	------	-------

Tableau 16 — Évolution de l'entraînement Seq2Seq sur 6 époques (*teacher_forcing_ratio* = 0,5).

Incident technique — La perplexité de validation chute à 1,01 dès la 4e époque, c'est-à-dire que le modèle prédit quasiment avec certitude le mot suivant correct. Cette convergence extrêmement rapide n'indique pas une performance de traduction généralisable, mais un surapprentissage attendu et inévitable compte tenu de la taille et de la répétitivité du corpus de repli (seulement 7 phrases distinctes, vues des centaines de fois à l'identique pendant l'entraînement comme en validation).

3.15 Évaluation : décodage glouton vs beam search, score BLEU

Les traductions générées sur le jeu de test reproduisent exactement les références, pour le décodage glouton comme pour le beam search ($k=3$), par exemple : « this is my house . » → « c'est ma maison . » (identique à la référence), ou « we are going to the market . » → « nous allons au marché . » (identique à la référence).

Stratégie de décodage	Score BLEU moyen (50 phrases de test)
Décodage glouton	0,9491
Beam search ($k = 3$)	0,9491

Tableau 17 — Score BLEU moyen sur le jeu de test, décodage glouton vs beam search.

Incident technique — Le score BLEU exceptionnellement élevé (0,95) et strictement identique entre glouton et beam search ne reflète pas une performance de traduction généralisable : il résulte directement du surapprentissage du corpus de repli évoqué en section 3.14. Avec seulement 7 phrases distinctes vues de très nombreuses fois, le modèle a mémorisé des correspondances quasi exactes plutôt qu'appris une compétence de traduction généralisable à des phrases nouvelles. L'égalité parfaite entre les deux stratégies de décodage s'explique également par cette mémorisation : lorsque le modèle est extrêmement confiant sur le mot suivant (probabilité proche de 1), le glouton et le beam search convergent nécessairement vers la même séquence, ce qui ne permet pas de mettre en évidence l'avantage théorique du beam search sur ce volet.

3.16 Synthèse — Volet B

Sur le plan méthodologique, l'ensemble du pipeline Seq2Seq (prétraitement bilingue, encodeur-décodeur GRU, teacher forcing, décodage glouton et beam search, calcul du score BLEU) a été implémenté et s'exécute correctement de bout en bout. Sur le plan des résultats, l'incident de téléchargement du corpus Tatoeba constitue ici la limite la plus significative du projet : le corpus de repli, bien que fonctionnel pour valider le code, ne permet pas d'évaluer la capacité de généralisation réelle du système ni de mettre en évidence l'avantage attendu du beam search sur le décodage glouton. Une réexécution avec le corpus Tatoeba complet (plusieurs milliers de paires distinctes), par exemple via une vérification manuelle de l'URL de téléchargement ou un téléchargement préalable et un chargement local, serait nécessaire pour obtenir des conclusions quantitatives généralisables.

3.17 Synthèse finale — Partie III

Question posée — Dans quelle mesure les architectures récurrentes permettent-elles de modéliser efficacement une séquence réelle, et comment justifier le passage d'un RNN simple vers un LSTM/GRU puis vers un schéma encodeur-décodeur pour une tâche de génération ou de traduction ?

Les deux volets expérimentaux, malgré les incidents de données rencontrés, confirment la validité du cadre théorique. Le passage du RNN simple vers LSTM/GRU se justifie par le chemin additif de mémoire de ces dernières architectures, qui limite structurellement le problème du gradient évanescent inhérent au BPTT ; sur le Volet A, le GRU obtient la meilleure perplexité de test avec un nombre de paramètres intermédiaire, illustrant le bon compromis performance/coût régulièrement rapporté dans la littérature pour les corpus de taille modeste. Le passage vers un schéma encodeur-décodeur se justifie dès que l'alignement entre entrée et sortie n'est plus direct, comme en traduction : l'encodeur compresse la source en un vecteur de contexte, le décodeur génère la cible de façon autoregressive, le teacher forcing rendant cet entraînement traitable en pratique. Les stratégies de décodage (glouton, beam search) déterminent ensuite la qualité de l'inférence — bien que leur différenciation n'ait pas pu être observée empiriquement ici en raison du corpus de repli très restreint.

Cette partie illustre enfin un enseignement transversal essentiel, indépendant des architectures testées : la qualité d'un système de deep learning est strictement bornée par la qualité des données qui l'alimentent. Les deux incidents de téléchargement rencontrés (corpus Gutenberg renvoyant une page 404, corpus Tatoeba mal téléchargé) ont eu un impact direct et démontrable sur les résultats (texte généré reproduisant du HTML, score BLEU artificiellement proche de 1 par mémorisation), alors même que l'implémentation des architectures elles-mêmes était correcte. Une limite commune aux deux volets, indépendante de ces incidents, reste le goulot d'étranglement informationnel du vecteur de contexte de taille fixe — limite historiquement résolue par les mécanismes d'attention et les architectures Transformer, qui dépassent le périmètre de ce projet mais en constituent la suite logique.

Synthèse générale et conclusion du projet

Question transversale finale (sujet) — Comment le deep learning adapte-t-il ses architectures à la structure des données — tabulaire, image et séquentielle — et pourquoi un même paradigme d'apprentissage supervisé doit-il être décliné différemment selon la géométrie, la dépendance locale, la temporalité et la représentation des données ?

Tableau comparatif des trois familles d'architectures

Critère	MLP (Partie I)	CNN (Partie II)	RNN/LSTM/GRU/Seq2Seq (Partie III)
Type de données	Tabulaire (variables hétérogènes)	Image (grille de pixels 2D)	Séquence (texte, longueur variable)
A priori structurel exploité	Aucun (variables indépendantes)	Localité + partage des poids	Ordre temporel + mémoire
Métrique principale observée	F1-score = 0,699 (test)	Exactitude = 0,854 (CNN, test)	Perplexité = 7,19 (GRU, test)
Paramètres du meilleur modèle	≈ 6 800 (MLP 66→64→32→1)	61 706 (LeNet)	190 160 (GRU, Volet A)
Principale limite observée	Surapprentissage dès l'époque 10-15	Peu d'époques, gains du CNN non maximisés	Incidents de données (corpus corrompus/répétitifs)
Limite structurelle de l'architecture	Aucune notion d'interaction spatiale/temporelle	Vision locale, pas de dépendance à longue portée native	Vecteur de contexte de taille fixe (Seq2Seq)

Tableau 18 — Tableau comparatif synthétique des trois familles d'architectures étudiées dans le projet.

Discussion transversale

Les trois parties de ce projet illustrent un même principe directeur : la performance et l'efficacité paramétrique d'un modèle de deep learning dépendent directement de l'adéquation entre la structure de son architecture et la structure statistique des données qu'il traite. Pour les données tabulaires (Partie I), aucune relation de proximité n'existe a priori entre les colonnes encodées : le MLP doit donc apprendre depuis zéro, par la seule force de ses couches denses, toutes les interactions utiles entre variables — ce qui le rend flexible mais peu efficace en nombre de paramètres et sujet au surapprentissage en l'absence de régularisation adaptée.

Pour les images (Partie II), la structure spatiale est au contraire extrêmement informative : deux pixels voisins sont presque toujours corrélés, et un même motif visuel (un bord, une texture) peut apparaître à n'importe quelle position de l'image. Le CNN encode directement ces deux propriétés — localité et invariance par translation — via les filtres convolutionnels et le partage des poids, ce qui lui permet d'atteindre une performance comparable à un MLP avec près de quatre fois moins de paramètres (61 706 contre 235 146 sur Fashion-MNIST), comme démontré empiriquement en section 2.9.

Pour les séquences textuelles (Partie III), c'est la dépendance temporelle ordonnée qui prime : la probabilité d'un mot dépend de ceux qui le précèdent, et cette dépendance peut s'étendre sur de

longues distances. Les architectures récurrentes (RNN, LSTM, GRU) encodent cette structure via un état caché transmis pas à pas, les portes de LSTM/GRU permettant de surcroît de limiter le problème du gradient évanescant inhérent au BPTT. Lorsque la tâche impose en plus un changement de longueur et de représentation entre l'entrée et la sortie (traduction), l'architecture doit encore se spécialiser davantage, via le découplage encodeur-décodeur, qui sépare explicitement la phase de compréhension de la source de la phase de génération de la cible.

Ce principe — la structure des données doit guider le choix de l'architecture, et non l'inverse — explique également pourquoi un même paradigme d'apprentissage supervisé (minimisation d'une perte par descente de gradient) produit des familles de modèles aussi différentes en apparence : la fonction objectif et l'algorithme d'optimisation restent les mêmes à travers les trois parties (entropie croisée ou perte binaire, optimiseur Adam, rétropropagation), mais la façon de calculer la prédiction à partir de l'entrée — c'est-à-dire l'architecture elle-même — doit nécessairement refléter la géométrie (grille 2D pour l'image), la temporalité (ordre pour le texte) ou l'absence de structure a priori (tabulaire) des données traitées.

Limites globales et perspectives

Au-delà des limites spécifiques à chaque partie (détaillées dans les sections d'analyse critique correspondantes), deux limites transversales se dégagent de l'ensemble du projet. Premièrement, plusieurs expérimentations ont été conduites avec un nombre d'époques volontairement réduit (3 à 15 selon les parties) pour des raisons de temps de calcul, ce qui ne permet pas toujours d'observer la convergence complète des modèles ni de conclure de façon définitive sur les écarts de performance entre architectures. Deuxièmement, et plus spécifiquement pour la Partie III, deux incidents de téléchargement de corpus ont significativement limité la portée des conclusions empiriques de cette partie, illustrant a posteriori un enseignement à part entière du projet : en deep learning, la robustesse du pipeline de données (vérification de l'intégrité des sources téléchargées, contrôles de cohérence a priori) est tout aussi déterminante pour la qualité des résultats que le choix de l'architecture elle-même.

Comme perspective commune aux trois parties, les architectures actuelles de référence (gradient boosting ou modèles de type Transformer pour le tabulaire, CNN profonds ou Vision Transformers pour l'image, et Transformers à base d'attention pour le texte) prolongent et dépassent les architectures étudiées ici, tout en s'appuyant sur les mêmes principes fondamentaux explorés dans ce projet : exploitation d'un a priori structurel adapté aux données, et résolution explicite des limites de mémoire/contexte (notamment via les mécanismes d'attention, qui généralisent et remplacent le vecteur de contexte unique du Seq2Seq récurrent étudié en Partie III).